Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#05)

Use of Venus Simulator for Register Level Execution of RISC-V Control Instructions

Student Name: *Syed Sikandar Abbas* Roll No: *24I-6553* Section: *CE-B*

Date performed: *16th September*, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

In this task, we were required to find out which instructions are actual instructions and which are pseudo-instructions, from table 1 and table 2. All the instructions that are written in assembly language and can be compiled, have a syntax, are all called actual instructions. They have their own op-codes which interact directly with hardware. Psuedo-instructions are for human understanding. All the comments in table 1 and 2, starting with # are called pseudo-instructions

Code:

Branches

type	target	# conditional branch to program label target
beq	t0, t1, target	# branch to target if t0 = t1
blt	t0, t1, target	# branch to target if t0 < t1
ble	t0, t1, target	# branch to target if t0 <= t1
bgt	t0, t1, target	# branch to target if t0 > t1
bge	t0, t1, target	# branch to target if t0 >= t1
bne	t0, t1, target	# branch to target if t0 <> t1

Table 1

Jumps

j	target	# unconditional jump to target
jal	ra, target	# unconditional jump to PC+target and save PC in ra
jalr	zero, const(ra)	# unconditional jump to ra+const and save PC in zero

Table 2

Output:

Actual Instructions:	**Psuedo-Instructions:**
beq t0, t1, target blt t0, t1, target ble t0, t1, target bgt t0, t1, target bge t0, t1, target bne t0, t1, target *From Table 1* j target jal ra, target jalr zero, const(ra) *From Table 2*	# branch to target if t0 = t1 # branch to target if t0 < t1 # branch to target if t0 <= t1 # branch to target if t0 > t1 # branch to target if t0 >= t1 # branch to target if t0 <> t1 *From Table 1* # unconditional jump to target # unconditional jump to PC+target and save PC in ra # unconditional jump to ra+const and save PC in zero *From Table 2*

Task 2:

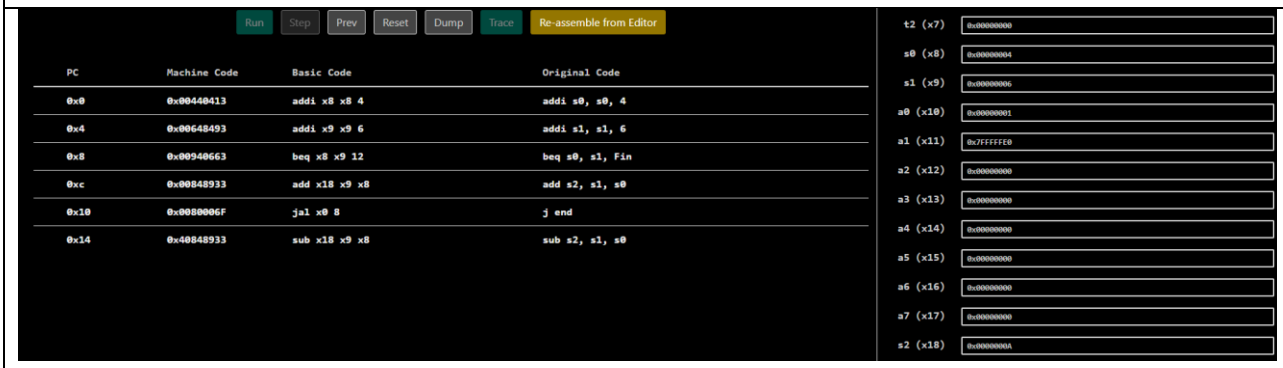
Description:

This task was supposed to write a code using beq and j instructions. The written code checks if s0 and s1 are equal. If they are, add s1 and s0 and jump to end. If not, subtract s0 from s1 and store in s2.

Code:

```
.data
.text
.globl main
    addi        s0,     s0,     4
        addi    s1,     s1,     6
        beq     s0,     s1,     Fin
True:
        add     s2,     s1,     s0
        j       end
Fin:
        sub     s2,     s1,     s0
end:
```

Output:



Task 3_1:

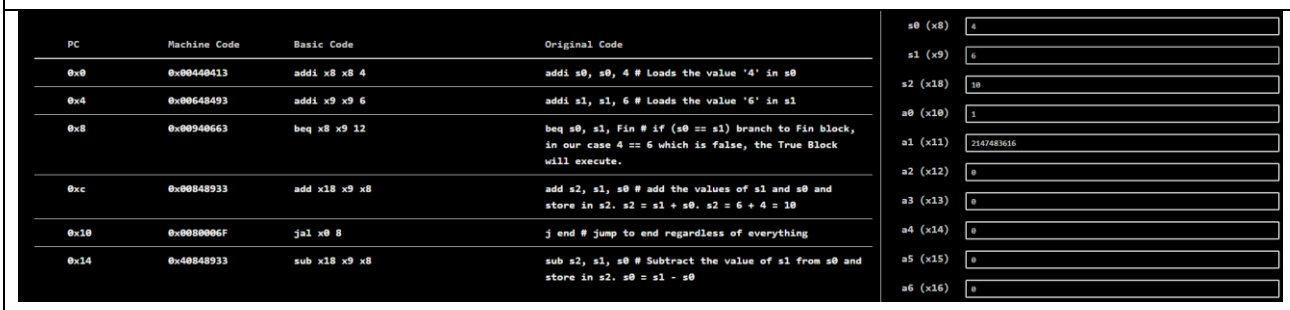
Description:

This task is a subset of task3. It provides us with a code which is supposed to be explained line by line which is done in the comments against each line in the code itself.

Code:

```
    addi s0, s0, 4 # Loads the value '4' in s0
    addi s1, s1, 6 # Loads the value '6' in s1
    beq s0, s1, Fin # if (s0 == s1) branch to Fin, in our case 4 == 6 that's false, the True
                      Block will execute.
True: # Start of True block
    add s2, s1, s0 # add the values of s1 and s0 and store in s2. s2 = s1 + s0. s2 = 6 + 4 = 10
    j end # jump to end regardless of everything
Fin: # Start of Fin block
    sub s2, s1, s0 # Subtract the value of s1 from s0 and store in s2. s0 = s1 - s0
    end: #End of the code
```

Output:



PC	Machine Code	Basic Code	Original Code
0x0	0x00440413	addi x8 x8 4	addi s0, s0, 4 # Loads the value '4' in s0
0x4	0x00648493	addi x9 x9 6	addi s1, s1, 6 # Loads the value '6' in s1
0x8	0x00940663	beq x8 x9 12	beq s0, s1, Fin # if (s0 == s1) branch to Fin block, in our case 4 == 6 which is false, the True Block will execute.
0xc	0x00848933	add x18 x9 x8	add s2, s1, s0 # add the values of s1 and s0 and store in s2. s2 = s1 + s0. s2 = 6 + 4 = 10
0x10	0x0080006F	jal x0 8	j end # jump to end regardless of everything
0x14	0x40848933	sub x18 x9 x8	sub s2, s1, s0 # Subtract the value of s1 from s0 and store in s2. s0 = s1 - s0

s0 (x8)	4
s1 (x9)	6
s2 (x18)	10
a0 (x10)	1
a1 (x11)	2147483616
a2 (x12)	0
a3 (x13)	0
a4 (x14)	0
a5 (x15)	0
a6 (x16)	0

Task 3_2:

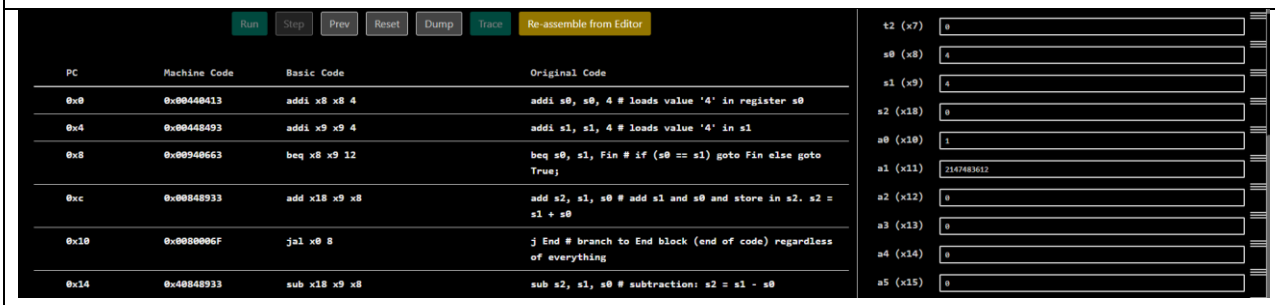
Description:

This task is a subset of task3. It provides us with a code which is supposed to be explained line by line which is done in the comments against each line in the code itself.

Code:

```
       addi s0, s0, 4 # loads value '4' in register s0
       addi s1, s1, 4 # loads value '4' in s1
       beq s0, s1, Fin # if (s0 == s1) goto Fin else goto True;
True: # Start of True block
       add s2, s1, s0 # add s1 and s0 and store in s2. s2 = s1 + s0
       j end # branch to End block (end of code) regardless of everything
Fin: # Start of Fin block
       sub s2, s1, s0 # subtraction: s2 = s1 - s0
End: # End of code
```

Output:



Task 4:

Description:

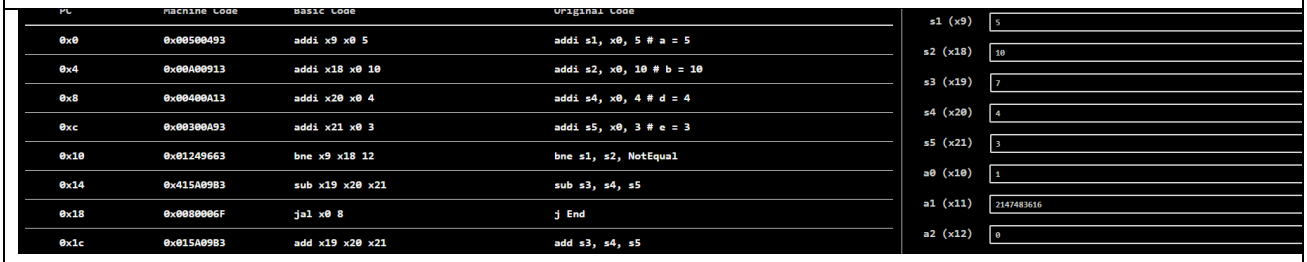
In this task, we were given a C program and asked to convert it into its equivalent assembly code.

Code:

```
.data
.text

.globl main
    addi s1, x0, 5 # a = 5
    addi s2, x0, 10 # b = 10
    addi s4, x0, 4 # d = 4
    addi s5, x0, 3 # e = 3
    bne s1, s2, NotEqual
Equal:
    sub s3, s4, s5
    j End
NotEqual:
    add s3, s4, s5
End:
```

Output:



PC	Machine Code	Basic Code	Original Code
0x0	0x00500493	addi x9 x0 5	addi s1, x0, 5 # a = 5
0x4	0x00A00913	addi x18 x0 10	addi s2, x0, 10 # b = 10
0x8	0x00400A13	addi x20 x0 4	addi s4, x0, 4 # d = 4
0xc	0x00300A93	addi x21 x0 3	addi s5, x0, 3 # e = 3
0x10	0x01249663	bne x9 x18 12	bne s1, s2, NotEqual
0x14	0x415A09B3	sub x19 x20 x21	sub s3, s4, s5
0x18	0x0080006F	jal x0 8	j End
0x1c	0x015A09B3	add x19 x20 x21	add s3, s4, s5

s1 (x9)	5
s2 (x18)	10
s3 (x19)	7
s4 (x20)	4
s5 (x21)	3
a0 (x10)	1
a1 (x11)	2147483616
a2 (x12)	0

Task 5:

Description:

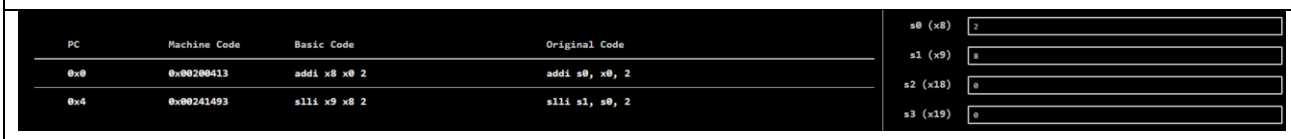
In this task, we were required to write a code that shifts a number by 2 and simulate it. We stored it in a saved register s0 and shifted that register by 2 and stored the result in another saved register s1, effectively multiplying the register s0's value by 4. *[2 * 4 = 8]*

Code:

```
.data
.text

.globl main
    addi s0, x0, 2
    slli s1, s0, 2
```

Output:



Task 6:

Description:

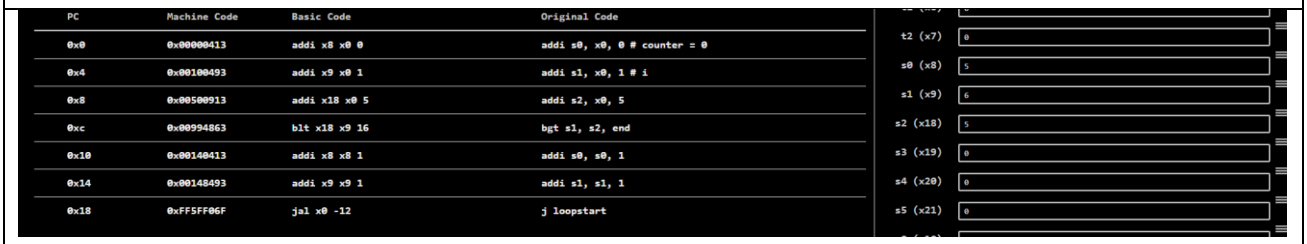
In this task, we were given a pseudo-code resembling C-syntax. It was a for loop based code and its assembly equivalent code was to be written and simulated in venus.

Code:

```
.data
.text

.globl main
    addi s0, x0, 0 # counter = 0
    addi s1, x0, 1 # i
    addi s2, x0, 5
for:
    bgt s1, s2, end
    addi s0, s0, 1
    addi s1, s1, 1
    j for
end:
```

Output:



PC	Machine Code	Basic Code	Original Code
0x0	0x00000413	addi x8 x0 0	addi s0, x0, 0 # counter = 0
0x4	0x00100493	addi x9 x0 1	addi s1, x0, 1 # i
0x8	0x00500913	addi x18 x0 5	addi s2, x0, 5
0xc	0x00994863	blt x18 x9 16	bgt s1, s2, end
0x10	0x00140413	addi x8 x8 1	addi s0, s0, 1
0x14	0x00148493	addi x9 x9 1	addi s1, s1, 1
0x18	0xFF5FF06F	jal x0 -12	j loopstart

t2 (x7)	0
s0 (x8)	5
s1 (x9)	6
s2 (x18)	5
s3 (x19)	0
s4 (x20)	0
s5 (x21)	0

Task 7:

Description:

We were given a C-based while loop code that takes an array of integers, starts at the base of array and increments each array value by its index + 5. And stops at index 5. The original C code was a while-loop based code which didn't have any line to increment the index. In case if the original code was run, it would be an infinite loop that kept on storing 0 + 5 = 5 in num[0] infinitely and would take us nowhere. I added the line by myself in the assembly code that cleanly runs the logic and stops when the loop condition becomes false.

Code:

```
.data
num: .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
.text
.globl main
main:
    li s2, 0 # i
    li s3, 5 # max
WHILE:
    bge s2, s3, ENDWHILE
        addi s4, s2, 5 # value to be stored
        la s5, num # go to the target location eg num[i]
        li t1, 4
        mul s6, s2, t1 # s6 = s2*t1 = i*4: to calculate the offset
        add s5, s5, s6 # actual offset
        sw s4, 0(s5) # num[i] = i + 5
        addi s2, s2, 1 #this line was added to make the code work. It was not in the given C
                       #snippet. Without this it would have been an infinite loop. C: i++;
        j WHILE
ENDWHILE:
```

Output:

Address	+3	+2	+1	+0
0x10000018	00	00	00	00
0x10000014	00	00	00	00
0x10000010	00	00	00	09
0x1000000C	00	00	00	08
0x10000008	00	00	00	07
0x10000004	00	00	00	06
0x10000000	00	00	00	05
0x0FFFFFFC	00	00	00	00
0x0FFFFFF8	00	00	00	00
0x0FFFFFF4	00	00	00	00
0x0FFFFFF0	00	00	00	00
0x0FFFFFEC	00	00	00	00
0x0FFFFFE8	00	00	00	00

0x0	0x00000913	addi x18 x0 0	li s2, 0 # i
0x4	0x00500993	addi x19 x0 5	li s3, 5 # max
0x8	0x03395463	bge x18 x19 40	bge s2, s3, ENDWHILE
0xc	0x00590A13	addi x20 x18 5	addi s4, s2, 5 # value to be stored
0x10	0x10000A97	auipc x21 65536	la s5, num # go to the target location eg num[i]
0x14	0xFF0A8A93	addi x21 x21 -16	la s5, num # go to the target location eg num[i]
0x18	0x00400313	addi x6 x0 4	li t1, 4
0x1c	0x02690B33	mul x22 x18 x6	mul s6, s2, t1 # s6 = s2*t1 = i*4: to calculate the offset
0x20	0x016A8AB3	add x21 x21 x22	add s5, s5, s6 # actual offset
0x24	0x014AA023	sw x20 0(x21)	sw s4, 0(s5) # num[i] = i + 5
0x28	0x00190913	addi x18 x18 1	addi s2, s2, 1
0x2c	0xFDDFF06F	jal x0 -36	j WHILE

t1 (x6)	0x00000004
s2 (x18)	0x00000005
s3 (x19)	0x00000005
s4 (x20)	0x00000009
s5 (x21)	0x10000010
s6 (x22)	0x00000010
s7 (x23)	0x00000000
s8 (x24)	0x00000000
s9 (x25)	0x00000000
s10 (x26)	0x00000000
s11 (x27)	0x00000000
t3 (x28)	0x00000000

Task 8:

Description:

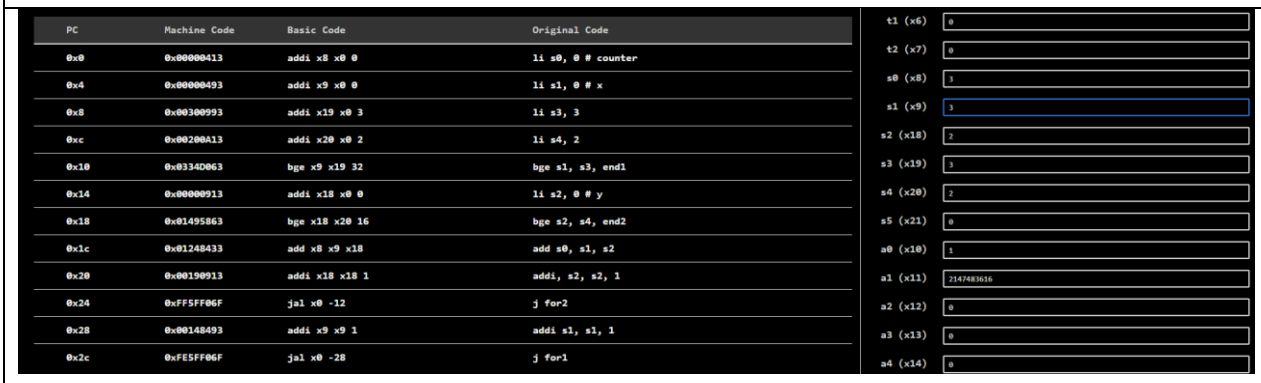
A nested loop C code was to be converted to RISC-V assembly and simulated using Venus.

Code:

```
.data
.text

.globl main
    li s0, 0 # counter
    li s1, 0 # x
    li s3, 3
    li s4, 2
    for1:
        bge s1, s3, end1
        li s2, 0 # y
        for2:
            bge s2, s4, end2
            add s0, s1, s2
            addi, s2, s2, 1
            j for2
        end2:
        addi s1, s1, 1
        j for1
    end1:
```

Output:



PC	Machine Code	Basic Code	Original Code
0x0	0x00000413	addi x8 x0 0	li s0, 0 # counter
0x4	0x00000493	addi x9 x0 0	li s1, 0 # x
0x8	0x00300993	addi x19 x0 3	li s3, 3
0xc	0x00200A13	addi x20 x0 2	li s4, 2
0x10	0x0334D063	bge x9 x19 32	bge s1, s3, end1
0x14	0x00000913	addi x18 x0 0	li s2, 0 # y
0x18	0x01495863	bge x18 x20 16	bge s2, s4, end2
0x1c	0x01248433	add x8 x9 x18	add s0, s1, s2
0x20	0x00190913	addi x18 x18 1	addi, s2, s2, 1
0x24	0xFF5FF06F	jal x0 -12	j for2
0x28	0x00148493	addi x9 x9 1	addi s1, s1, 1
0x2c	0xFE5FF06F	jal x0 -28	j for1

t1 (x6)	0
t2 (x7)	0
s0 (x8)	3
s1 (x9)	3
s2 (x18)	2
s3 (x19)	3
s4 (x20)	2
s5 (x21)	0
a0 (x10)	1
a1 (x11)	2147483616
a2 (x12)	0
a3 (x13)	0
a4 (x14)	0